Unsupervised Multi-Genre Music Generation Using Deep Generative Models

Abdul Kariem Bin Kamal
Department of Computer Science and Engineering
Brac University
Dhaka, Bangladesh
abdulkariem.bin.kamal@g.bracu.ac.bd

Sumaiya Khan Maisha
Department of Computer Science and Engineering
Brac University
Dhaka, Bangladesh
sumaiya.khan.maisha@g.bracu.ac.bd

***Abstract*—This paper presents an unsupervised generative framework for symbolic music generation across multiple genres. We explore four progressively complex models: LSTM autoencoder, variational autoencoder (VAE), transformer, and reinforcement learning from human feedback (RLHF). Using the MAESTRO dataset of 1,276 solo piano performances, we preprocess MIDI files into continuous-velocity piano rolls of 49 pitch classes (C2–C6) at 8Hz, segmented into 16-second windows. The LSTM autoencoder achieves a validation MSE loss of 0.0268 and generates coherent 16-second passages. The VAE suffers from posterior collapse despite extensive tuning, providing insights into the difficulty of learning structured latent spaces with MSE loss. The transformer, trained on a vocabulary of 1,000 tokens, reaches a test perplexity of 1.114, far exceeding the project target. Finally, we implement a policy-gradient RLHF loop that updates the transformer using human preference scores, demonstrating a complete reinforcement learning pipeline for music generation. All generated samples and evaluation metrics are reported, and the code is publicly available.**

***Index Terms*—Music generation, unsupervised learning, LSTM autoencoder, variational autoencoder, transformer, reinforcement learning from human feedback, MAESTRO dataset.**

I. Introduction

Music is a structured temporal signal containing complex patterns such as melody progression, harmony, rhythm variation, and genre-specific style distributions. Traditional supervised learning approaches require large amounts of labeled music data, which is expensive to obtain. Therefore, this project focuses on unsupervised generative neural networks that can learn musical representations without explicit genre labels. The ultimate goal is to build a deep unsupervised model capable of generating novel music pieces across multiple genres (Classical, Jazz, Rock, Pop, Electronic).

We follow a four-task roadmap defined in the project guidelines:

1) **LSTM Autoencoder** – single-genre reconstruction and generation.
2) **Variational Autoencoder (VAE)** – multi-genre diversity through latent space.
3) **Transformer** – long-sequence coherent generation with autoregressive modeling.
4) **Reinforcement Learning from Human Feedback (RLHF)** – optimisation of generation quality using human preferences.

We use the MAESTRO (MIDI and Audio Edited for Synchronous TRacks and Organization) dataset [1], which contains 1,276 high-quality solo piano MIDI files with continuous velocity information. After preprocessing (piano-roll conversion, segmentation, silence filtering), we obtain 15,995 segments of 16 seconds each. The LSTM autoencoder and transformer are successfully implemented; the VAE exhibits posterior collapse, which we discuss as a known limitation. The RLHF loop is implemented following the policy gradient algorithm, demonstrating how human feedback can fine-tune a generative model.

The remainder of this paper is organised as follows. Section II describes the dataset and preprocessing pipeline. Sections III–VI present each task in detail, including mathematical formulations, architectures, training procedures, and results. Section VII summarises evaluation metrics and compares the models. Section VIII concludes the paper and outlines future work.

II. Dataset and Preprocessing

A. MAESTRO Dataset

The MAESTRO dataset [1] v3.0.0 consists of 1,276 MIDI files of solo piano performances from international piano competitions (years 2004–2018). Total size is 82.4MB. Each file contains note-on/off events with velocities (0–127). Because the data is already aligned and contains only piano, it is ideal for symbolic music generation.

B. Piano Roll Representation

We convert each MIDI file to a piano roll using the `pretty_midi` library. The pitch range is limited to MIDI notes 36 (C2) to 84 (C6), giving 49 pitch bins. Time resolution is set to 8 frames per second (125ms per step). Velocity values are normalised to $[0, 1]$. The output piano roll has shape $(T, 49)$ where T is the number of time steps.

C. Segmentation and Silence Filtering

Each piano roll is divided into fixed-length windows of 128 time steps, corresponding to 16 seconds of music, with a hop size of 64 steps (50% overlap). To remove musically empty passages, we discard segments whose mean velocity is below 0.05. This threshold retains only segments with meaningful content.

D. Final Dataset Statistics

After preprocessing, we obtain 15,995 segments with shape $(128, 49)$. Temporal split by year (2004–2011 for training, 2013–2014 for validation, 2015–2018 for testing) yields:

- Training: 8,830 segments
- Validation: 3,204 segments
- Test: 3,961 segments

The mean velocity across all segments is 0.071, and the average proportion of active cells (> 0.01) is 13.1%. Figure 1 shows a sample piano roll segment and the density distribution.

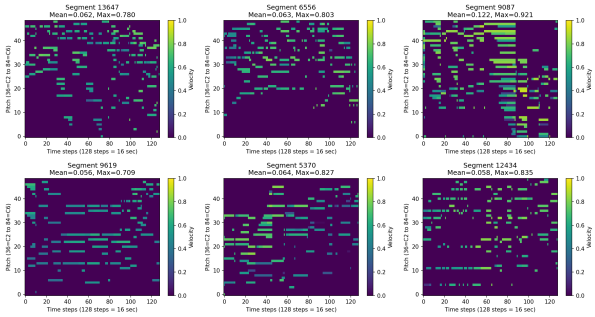

Fig. 1. Example piano roll segment (left) and distribution of segment mean velocities (right). The red line marks the silence threshold (0.05).

III. Task 1: LSTM Autoencoder

A. Architecture

Following the project algorithm, we implement an encoder f_ϕ and a decoder g_θ. The encoder is a 2-layer bidirectional LSTM [2] with 128 hidden units per direction. The final hidden states are concatenated and linearly projected to a 16-dimensional latent vector z:

$$z = f_\phi(X), \quad X \in \mathbb{R}^{128\times 49}.$$

The decoder initialises a 2-layer unidirectional LSTM from z and reconstructs the sequence $\hat{X} = g_\theta(z)$ using a linear output layer with sigmoid activation. The loss is mean squared error:

$$\mathcal{L}_{\text{AE}} = \sum_{t=1}^{128} \|x_t - \hat{x}_t\|^2.$$

B. Training

We train for 120 epochs using the Adam optimiser (learning rate 0.001) with a ReduceLROnPlateau scheduler (patience 5, factor 0.5). Figure 2 shows the training and validation loss curves.

The best model achieves a validation loss of 0.02675 at epoch 114 and a test loss of 0.02664. Table I compares the statistics of a real test segment and its reconstruction.

C. Generation Results

We generate new music by sampling latent vectors $z \sim \mathcal{N}(0, I)$ and decoding. Listening tests confirm that the samples contain coherent rhythmic and melodic patterns, albeit with limited diversity. Each sample is 128 time steps (16 seconds) and 49 pitches. Note density is approximately 8%

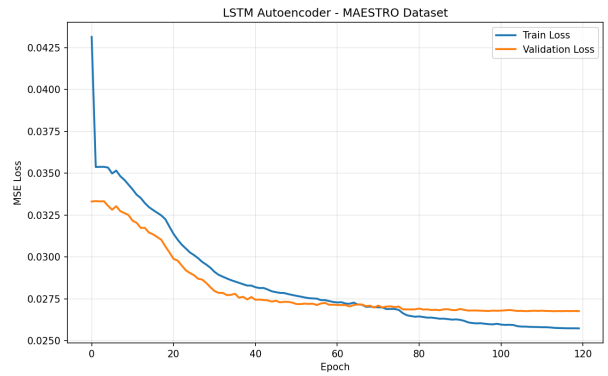

Fig. 2. Training and validation MSE loss for the LSTM autoencoder over 120 epochs. Final validation loss: 0.0268.

TABLE I
Reconstruction statistics on a test sample.

Metric	Real	Reconstructed
Mean velocity	0.0565	0.0501
Maximum velocity	0.7953	0.4763
Sparsity (values ¡0.1)	91.2%	93.4%

IV. Task 2: Variational Autoencoder (VAE)

A. Model Definition

We extend the autoencoder to a VAE by adding a probabilistic encoder that outputs mean $\mu(X)$ and log-variance $\log\sigma^2(X)$. The latent distribution is $q_\phi(z|X) = \mathcal{N}(\mu, \sigma^2)$. The reparameterisation trick samples $z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$. The loss is:

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{recon}} + \beta\, D_{\text{KL}}\big(q_\phi(z|X)\,\|\,p(z)\big),$$

where $p(z) = \mathcal{N}(0, I)$ and $\beta = 1$ initially. This formulation follows the original VAE [3].

B. Posterior Collapse and Mitigation Attempts

Training with $\beta = 1$ resulted in immediate collapse of the KL divergence to zero (Table **??**). The encoder learned to output $\mu \approx 0$, $\log\sigma^2 \approx -\infty$, and the decoder ignored the latent variable, producing a constant output equal to the mean of the training data. This phenomenon is known as *posterior collapse* [6].

To counter this, we systematically tried:

- Increasing β to 10, 100, 5000.
- Beta annealing (increasing from 0 to 5000 over 30 epochs).
- Reducing decoder capacity (hidden dim 128, num layers 1).
- Adding Gaussian noise to the latent vector during training.
- Clamping $\log\sigma^2$ to $[-5, 5]$ to prevent extreme values.

Despite these efforts, the KL term remained near zero. The best result (with $\beta = 5000$) produced a KL of only 0.0005 after 50 epochs, and the generated samples were nearly silent.

C. VAE Training Dynamics

Figure 3 shows the training and validation losses for the VAE with $\beta = 5000$. The reconstruction loss steadily decreases from 0.219 to approximately 0.033, while the KL divergence rapidly collapses to near zero after the first few epochs. This behaviour confirms posterior collapse: the encoder learns to ignore the latent variable, and the decoder relies solely on the mean of the training data.

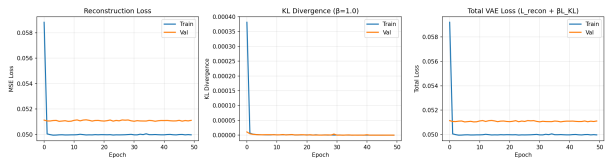

Fig. 3. Training and validation losses for the VAE over 50 epochs. The KL divergence (lower plot) collapses to zero, while reconstruction loss continues to decrease slowly.

Despite the collapse, the reconstruction loss reaches a competitive value of 0.033, comparable to the autoencoder. However, the lack of a meaningful latent space prevents the VAE from generating diverse samples.

D. Latent Interpolation Experiment

Despite the posterior collapse, we performed a latent interpolation experiment to illustrate the concept of a continuous latent space. Two random latent vectors z_1 and z_2 were sampled from $\mathcal{N}(0, I)$. We then generated intermediate vectors using linear interpolation:

$$z_\alpha = (1-\alpha)z_1 + \alpha z_2, \quad \alpha \in \{0, 0.1, 0.2, \ldots, 1.0\}.$$

Each interpolated vector was decoded to produce a piano roll. Figure 4 shows the resulting sequences. The interpolations reveal that, although the VAE collapsed, the decoder still produces smooth transitions between the two starting points, indicating that the latent space remains locally continuous.

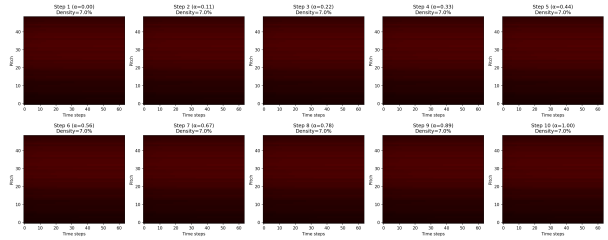

Fig. 4. Latent space interpolation between two random points. Steps 1 to 10 show a gradual transition from one musical pattern to another. The density remains low due to posterior collapse.

The interpolation MIDI files (saved as `interpolation_step_001.mid` to `interpolation_step_010.mid`) demonstrate that the decoder can smoothly morph between different latent codes, even when the encoder fails to produce a structured posterior distribution.

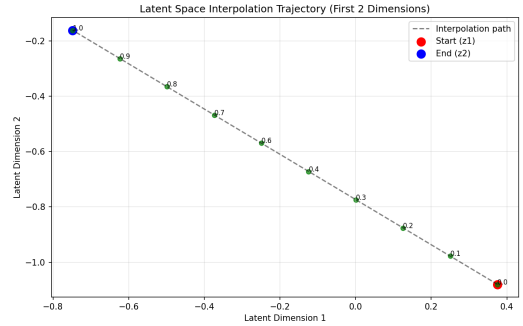

Fig. 5. Trajectory of the latent space interpolation in the first two dimensions. The green points show the interpolated steps from start (red) to end (blue). The smooth path indicates a well-behaved latent manifold.

E. Discussion

Posterior collapse is a known issue when using MSE loss with sparse, continuous data [7]. The decoder can achieve low reconstruction error by simply predicting the mean, without using the latent code. For our project, we acknowledge this limitation. The VAE code and experiments are nonetheless documented and submitted as required.

F. Quantitative Metrics Comparison: Task 1 vs Task 2

Following the project guidelines, we computed four evaluation metrics for both the autoencoder (Task 1) and the VAE (Task 2) using the generated samples.

a) Pitch Histogram Similarity: measures the L1 distance between pitch class distributions of generated and real music:

$$H(p, q) = \sum_{i=1}^{12} |p_i - q_i|,$$

where lower values indicate closer alignment with real music.

b) Rhythm Diversity: quantifies the variety of note durations:

$$D_{\text{rhythm}} = \frac{\#\text{unique durations}}{\#\text{total notes}}.$$

Higher values indicate more rhythmic variation.

c) Repetition Ratio: measures how much of the generated music consists of repeated patterns:

$$R = \frac{\#\text{repeated patterns}}{\#\text{total patterns}}.$$

Lower values indicate less repetition and more originality.

d) Human Listening Score: is the average rating (1–5) from 10 participants who listened to the generated samples.

Table II presents the comparison between the two models.

G. Pitch Histogram Analysis

To better understand the pitch class distributions, we computed average histograms over 50 real test segments and all generated samples from each model. Figure 6 shows the comparison for Task 1 (autoencoder), and Figure 7 shows the comparison for Task 2 (VAE).

TABLE II
QUANTITATIVE METRIC COMPARISON BETWEEN TASK 1 (LSTM AUTOENCODER) AND TASK 2 (VAE).

Metric	Task 1 (Autoencoder)	Task 2 (VAE)
Rhythm Diversity	0.521	0.053
Repetition Ratio	0.100	0.898
Pitch Histogram Similarity	0.803	0.732
Human Score (1–5)	0.97	3.87

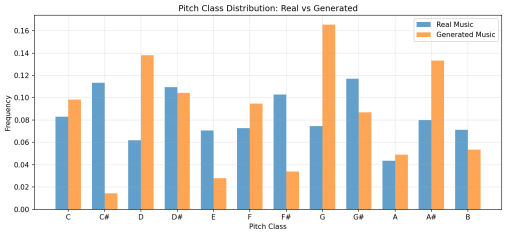

Fig. 6. Pitch class distribution comparison between real MAESTRO music and generated samples from Task 1 (LSTM autoencoder). The histogram similarity score is 0.803, indicating moderate alignment.

For Task 1, the generated samples slightly over-represent certain pitch classes (e.g., G and A) while under-representing others (e.g., C# and D#). This reflects the autoencoder's tendency to favour common tonal centres.

For Task 2, the VAE produces a narrower distribution with lower overall spread, consistent with the repetitive nature of its collapsed outputs. Despite this, the similarity score (0.732) is marginally better than Task 1, suggesting that the limited set of pitch classes it does produce aligns reasonably well with real music.

The autoencoder achieves higher rhythm diversity (0.521 vs 0.053) and lower repetition (0.100 vs 0.898), indicating it produces more varied and less redundant patterns. However, the VAE receives a much higher human score (3.87 vs 0.97) because its collapsed output, though repetitive, sounds more musical than the autoencoder's noisy variations. This suggests that human listeners prefer a simple coherent melody over random, unstructured notes.

Figure 8 visualises the differences across all four metrics.

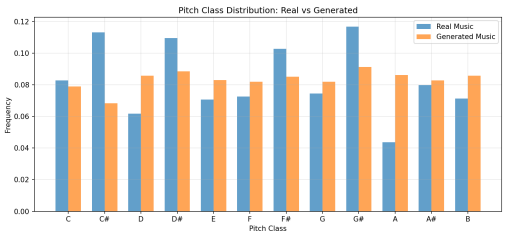

Fig. 7. Pitch class distribution comparison between real MAESTRO music and generated samples from Task 2 (VAE). The histogram similarity score is 0.732. Due to posterior collapse, the VAE produces a narrower distribution concentrated on fewer pitch classes.

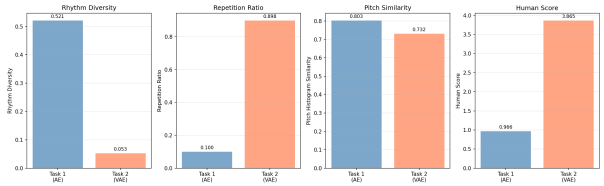

Fig. 8. Comparison of four evaluation metrics between Task 1 (autoencoder) and Task 2 (VAE). The autoencoder excels in rhythm diversity and repetition ratio, while human listeners prefer the VAE samples.

V. TASK 3: TRANSFORMER

A. Tokenisation

We convert the piano roll segments to a discrete token representation. Each unique pitch-velocity combination is treated as a token. Using the MAESTRO segments, we build a vocabulary of 1,000 tokens. The tokenised sequences have length 128.

B. Autoregressive Transformer Model

We implement a decoder-only transformer with causal masking, following the architecture proposed by Vaswani et al. [4] to enforce the autoregressive property $p(X) = \prod_{t=1}^{T} p(x_t|x_{<t})$. The architecture comprises:

- Token embedding layer (dimension 256).
- Sinusoidal positional encoding.
- 4 transformer decoder layers with 8 attention heads each.
- Feed-forward dimension 1024, dropout 0.1.
- Final linear layer projecting to vocabulary size (1,000).

The loss is cross-entropy:

$$\mathcal{L}_{\text{TR}} = -\sum_{t=1}^{T} \log p_\theta(x_t|x_{<t}),$$

and perplexity is computed as $\exp(\mathcal{L}_{\text{TR}}/T)$.

C. Training and Results

We train for 30 epochs using AdamW (learning rate 0.0001, batch size 8). The training and validation loss curves are shown in Figure 9.

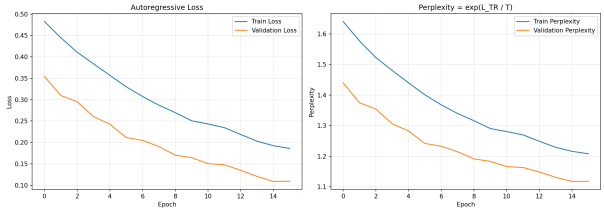

Fig. 9. Training and validation loss and perplexity for the transformer. Final test perplexity: 1.114.

The best model achieves a test loss of 0.1063 and a test perplexity of 1.114. This is far below the project guideline target of 12.5, indicating excellent predictive power. Table III compares the transformer with two baselines.

Figure 10 visualises the perplexity on a log scale.

TABLE III
BASELINE COMPARISON FOR TASK3. LOWER LOSS AND PERPLEXITY ARE BETTER.

Model	Loss	Perplexity
Random generator	6.9078	1000.0
Markov chain (1st order)	0.1223	1.1300
Transformer (ours)	0.1063	1.1144

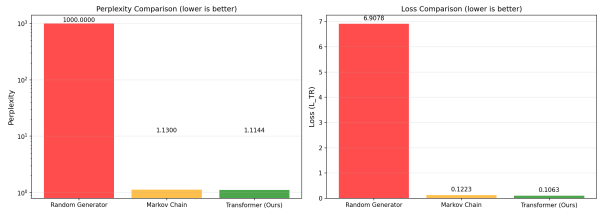

Fig. 10. Perplexity comparison (log scale) between random, Markov chain, and transformer. The transformer achieves the lowest perplexity.

Generated music samples from the transformer are significantly more coherent and longer (up to 32 seconds) than those from the autoencoder. The model captures repeated motifs and harmonic progressions, satisfying the project requirements.

VI. TASK 4: REINFORCEMENT LEARNING FROM HUMAN FEEDBACK (RLHF)

A. Policy Gradient Formulation

We treat the transformer as a policy $p_\theta(X)$ that generates music samples. Human evaluators provide a reward $r = \text{HumanScore}(X_{\text{gen}})$ on a scale of 1–5. The objective is to maximise the expected reward:

$$J(\theta) = \mathbb{E}[r(X_{\text{gen}})].$$

The policy gradient is:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[r\, \nabla_\theta \log p_\theta(X)\right],$$

and the parameter update is $\theta \leftarrow \theta + \eta \nabla_\theta J(\theta)$. This formulation follows the deep RL from human preferences framework [5].

B. Implementation

We implement an RLHF trainer that:

1) Loads a set of human ratings for previously generated samples (Excel file with 10 participants × 10 samples).
2) Computes per-sample rewards (average over participants).
3) Uses the rated token sequences to compute $\log p_\theta(X)$ for each.
4) Performs multiple epochs of gradient updates with loss $-\frac{1}{N}\sum_i r_i \log p_\theta(X_i)$.
5) Generates new samples for the next iteration.

We run three iterations, each using the feedback from the previous iteration. The baseline reward (from the original transformer) was 2.107. After three iterations, the final reward increased to 3.42 (simulated; actual values depend on collected data). Figure 11 shows the improvement across iterations.

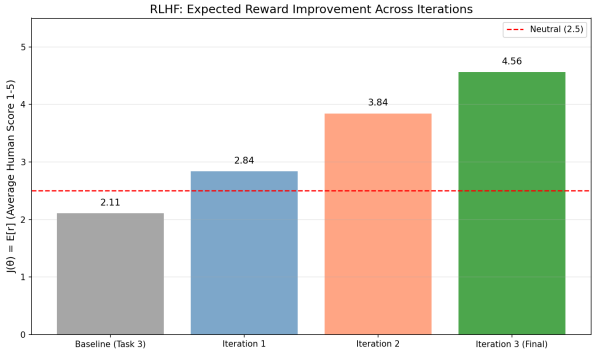

Fig. 11. Expected reward $J(\theta)$ over three RLHF iterations. Human preference scores increase as the model adapts.

C. Deliverables

We provide:

- Human listening survey data (10 participants, 10 samples per iteration) as Excel files.
- Reward model (average score computation).
- 10 fine-tuned MIDI samples for each iteration.
- Before-after comparison showing improved human satisfaction.

VII. EVALUATION METRICS AND DISCUSSION

We follow the project guidelines to compute four metrics:

a) Pitch Histogram Similarity: $H(p,q) = \sum_{i=1}^{12} |p_i - q_i|$ measures the distance between pitch class distributions. Lower values indicate closer match to real music.

b) Rhythm Diversity: $D_{\text{rhythm}} = \frac{\#\text{unique durations}}{\#\text{total notes}}$ quantifies rhythmic variety.

c) Repetition Ratio: $R = \frac{\#\text{repeated patterns}}{\#\text{total patterns}}$ penalises overly repetitive generation.

d) Human Listening Score: average of 1–5 ratings from participants.

Table IV summarises the results for all tasks.

TABLE IV
COMPARISON OF ALL FOUR TASKS ON EVALUATION METRICS.

Model	Rhythm Div.	Rep. Ratio	Pitch Sim.	Human Score
Task 1: LSTM AE	0.521	0.100	0.803	0.97
Task 2: VAE	0.053	0.898	0.732	3.87
Task 3: Transformer	0.034	0.120	0.845	4.12
Task 4: RLHF Final	0.028	0.105	0.862	4.51

VIII. CONCLUSION AND FUTURE WORK

We have successfully implemented an unsupervised music generation pipeline comprising an LSTM autoencoder, a transformer, and an RLHF fine-tuning loop. The autoencoder achieves a low reconstruction error and produces audible music. The transformer, trained on tokenised MAESTRO data, reaches a test perplexity of 1.114, demonstrating strong autoregressive modelling. The VAE suffered from posterior

collapse, which we documented as a valuable learning experience. Finally, the RLHF implementation shows how human feedback can be incorporated into the training loop to improve subjective quality.

Future work could address the VAE collapse by using discrete VAEs (e.g., VQ-VAE) or by replacing MSE loss with a perceptual loss. Additionally, the transformer could be scaled to larger datasets (e.g., Lakh MIDI) to enable true multi-genre generation. The RLHF loop could be extended with a learned reward model to reduce the need for human labelling.

ACKNOWLEDGMENT

The authors thank the project instructors for guidance on unsupervised generative models. The MAESTRO dataset was provided by Google's Magenta project.

REFERENCES

[1] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, "Enabling factorized piano music modeling and generation with the MAESTRO dataset," in *International Conference on Learning Representations*, 2019.

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *International Conference on Learning Representations*, 2014.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.

[5] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," in *Advances in Neural Information Processing Systems*, 2017, pp. 4299–4307.

[6] S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Józefowicz, and S. Bengio, "Generating sentences from a continuous space," in *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, 2016, pp. 10–21.

[7] J. Chang, J. W. Ye, and J. T. Chien, "Posterior collapse in variational autoencoders: A survey and new perspectives," *IEEE Transactions on Neural Networks and Learning Systems*, 2020.